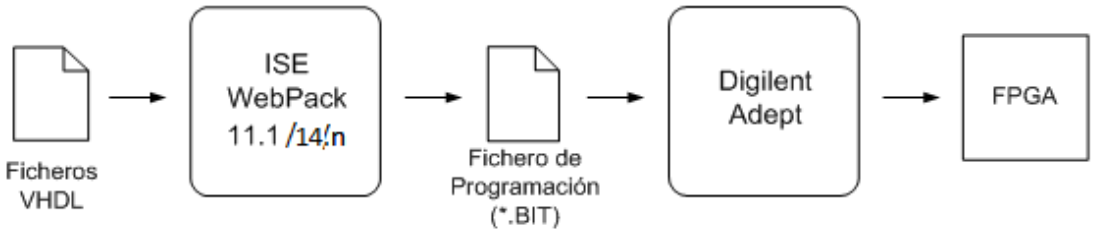
1. Introducción

Trabajaremos con una placa genérica (Nexys2 o Nexys3) que contiene periféricos conectados a la FPGA que podemos utilizar para realizar nuestros diseños. El fabricante (DIGILENT) proporciona una aplicación ADEPT para poder transferir ficheros de programación (*.BIT) a la FPGA.

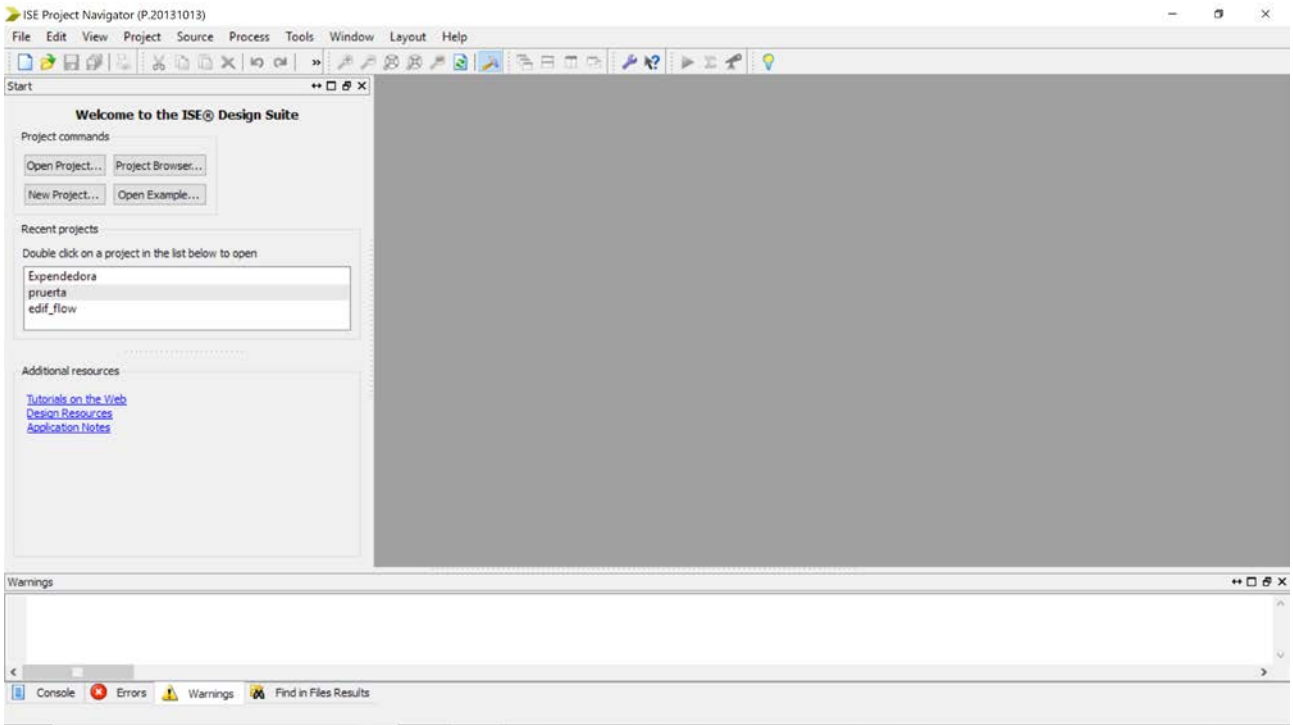
Partiremos de nuestro diseño creado con Mentor Graphics, que nos proporciona la descripción en VHDL de todos los módulos que lo componen.

Necesitamos herramientas que nos permitan generar el fichero de programación en base al diseño que hemos realizado en VHDL. En nuestro caso utilizaremos la herramienta ***ISE WebPack*** desarrollado por Xilinx. Los pasos para implementar un circuito especificado en VHDL en la FPGA son:

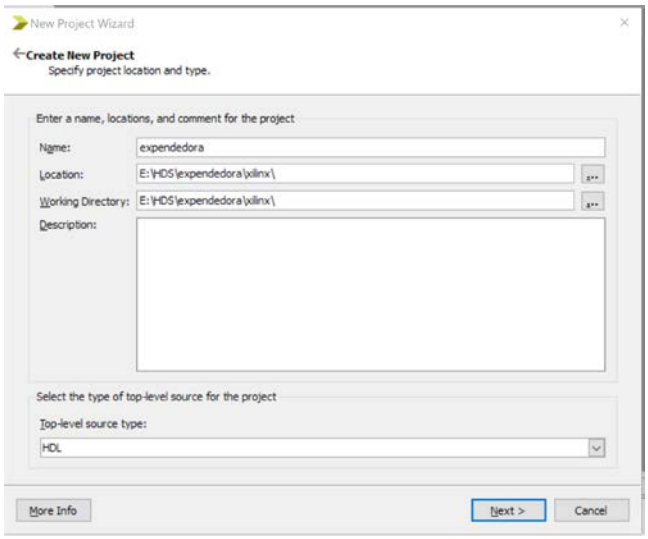


2. Creación de un proyecto nuevo en ISE

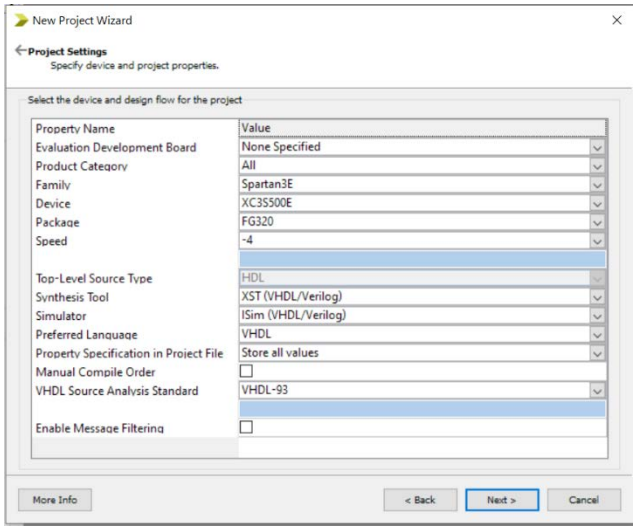
Abrimos el entorno de desarrollo *Xilinx ISE Desing* y seleccionamos la opción de nuevo proyecto.

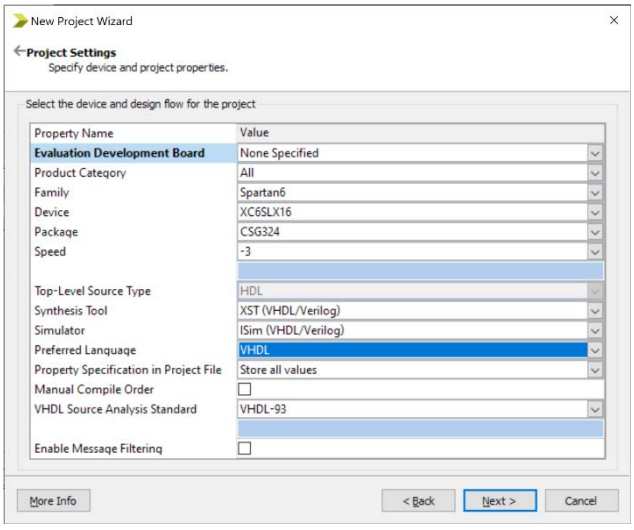


Se abrirá el asistente de creación de proyectos donde se debe introducir el nombre del proyecto, el directorio de trabajo y el lenguaje que vamos a utilizar para realizar los diseños. Si el directorio de trabajo no existe el asistente creará uno nuevo con el nombre especificado. No olvidar seleccionar HDL como “Top-Level Source Type”.



Seleccionamos el dispositivo que se va a utilizar. Se debe especificar la familia, el chip y el paquete que describe las características de la FPGA. De esta forma el entorno de desarrollo conoce las limitaciones y posibilidades del chip,etc.





Configuración para Nexys2.

Configuración para Nexys3.

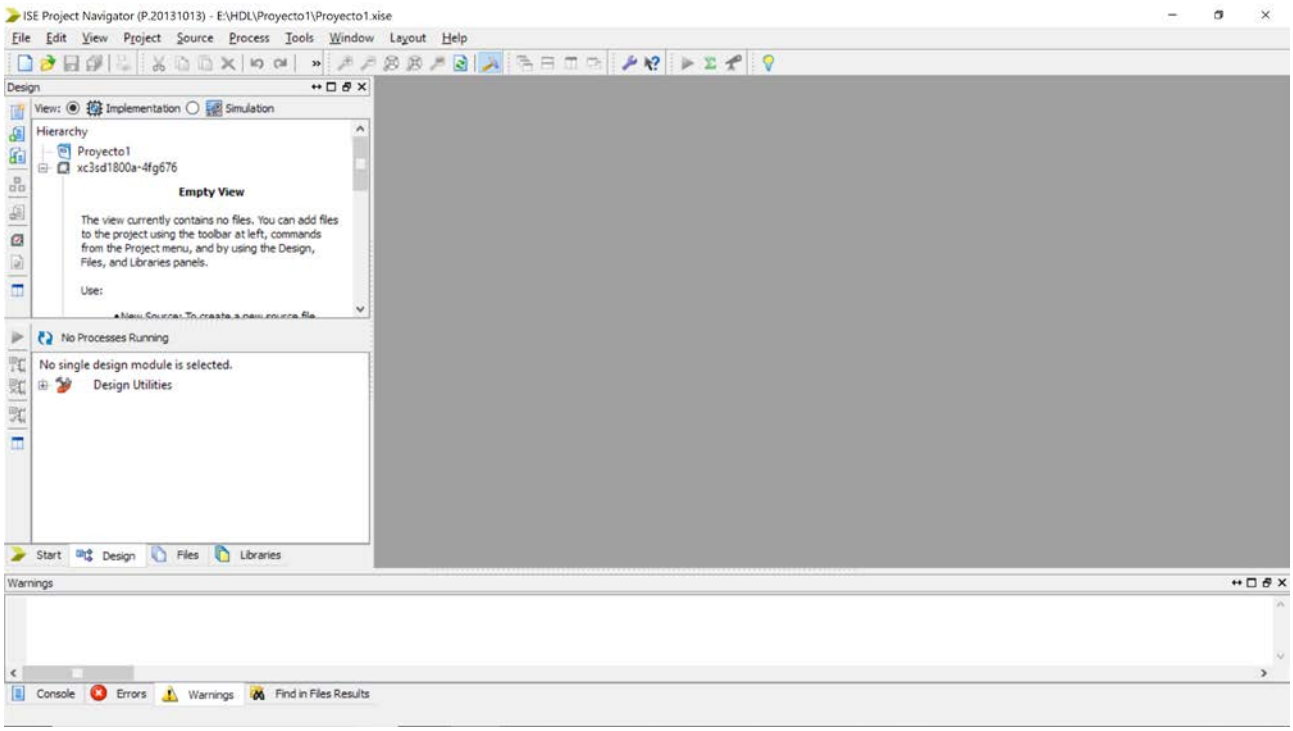
CUIDADO: hay alguna Nexys2 que usa como dispositivo el XC3S1200E en lugar del XC3S500E.

Una vez generado el proyecto, ISE abre el "Project Navigator" estando activa la pestaña Design. Aparecen las tres secciones marcadas:

1. Panel de diseño: En la parte superior (Hierarchy) aparece un listado de los ficheros que contiene el proyecto y en la parte inferior la lista de procesos que se pueden aplicar al fichero seleccionado. Las pestañas nos permiten navegar para ver un listado de ficheros en disco "Files" y las librerías creadas "Libraries". Las librerías son contenedores donde podemos almacenar los diseños realizados. Si vamos a la pestaña podemos observar que aparecen dos librerías "verilog" y "work" vacías. Por defecto, todos los diseños VHDL se almacenarán en la librería "work.

2. Editor: Permite modificar el código fuente que contiene el fichero seleccionado en el panel de diseño.

3. Consola: Muestra los resultados de la síntesis, mensajes de aviso, error e información.



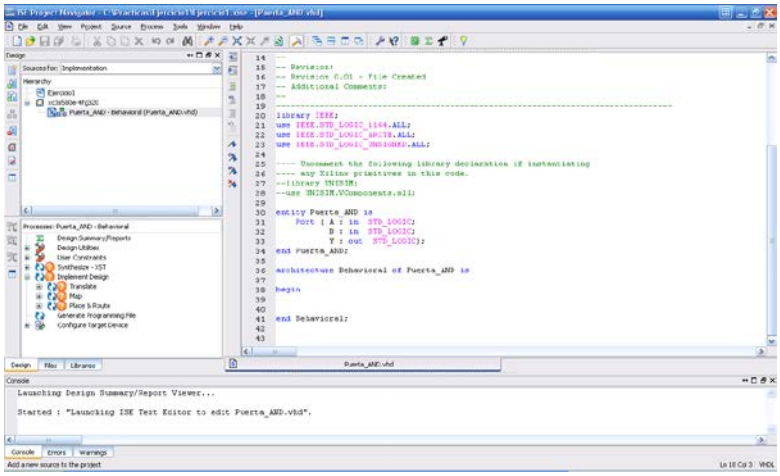
4. Añadir archivos VHDL al proyecyo ISE

En el panel de diseño (Hierarchy) aparecen dos ficheros, uno es el fichero del proyecto y el otro el dispositivo que vamos a programar.

Pulsando el botón derecho sobre el dispositivo en el panel de diseño, puede seleccionarse entre crear un nuevo fuente (*New source…*) o añadir un fuente VHDL ya escrito (*Add source…). N*osotros añadiremos los ficheros VHDL previamente creados con Mentor pero nunca debemos usar (*Add Copy of source..*) ya que debemos usar el fichero original creado por Mentor por si posteriormente queremos realizar una modificación desde Mentor, dicha modificación estará actualizada en el proyecto ISE. También podemos usar este icono  para añadir ficheros.

Los ficheros VHDL tendrán la extensión *.vhd* y estarán dentro del proyecto Mentor, en el directorio *hdl* de la librería de dicho proyecto. Si tenemos un diseño jerárquico compuesto por diversos elementos, debemos incluir los fichero VHDL de todos ellos. Mentor incluye la sentencia *LIBRARY Nombre_de_libreria_Mentor;* en el fichero del nivel jerárquico superior. Esto nos dará error en el ISE por lo que debemos borrar esa línea de código cada vez que Mentor regenere dicho fichero (lo podemos hacer desde el propio ISE).

Cuando termina la creación del fichero ISE lo añade al proyecto y si lo seleccionamos en la ventana de diseño podemos ver el código fuente VHDL generado para la entidad y el esqueleto de la arquitectura para continuar editándola.

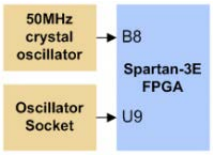


5. Definir el fichero de conexiones y restricciones.

Debemos definir el fichero de restricciones "User Constraint File" (*.UCF). Este fichero le indica al entorno de desarrollo, entre otras cosas, qué pines físicos de la FPGA se corresponden con las señales de entrada/salida de la entidad.

En la placa Nexys2 disponemos de una serie de entradas y salidas accesibles desde la FPGA:

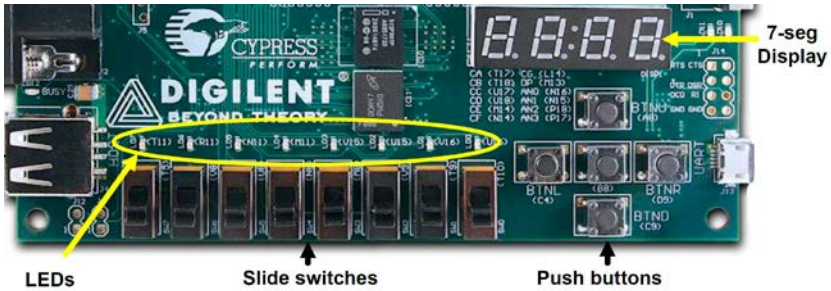
8 switches
4 pulsadores
8 leds
4 dígitos 7 segmentos
1 reloj soldado en placa de 50 MHz (pin B8)
1 slot para conectar un reloj de la frecuencia deseada (pin U9)





En la placa Nexys3 disponemos de una serie de entradas y salidas accesibles desde la FPGA:

8 switches
4 pulsadores
8 leds
4 dígitos 7 segmentos
1 reloj soldado en placa de 100 MHz (pin V10)
1 slot para conectar un reloj de la frecuencia deseada (pin U9)

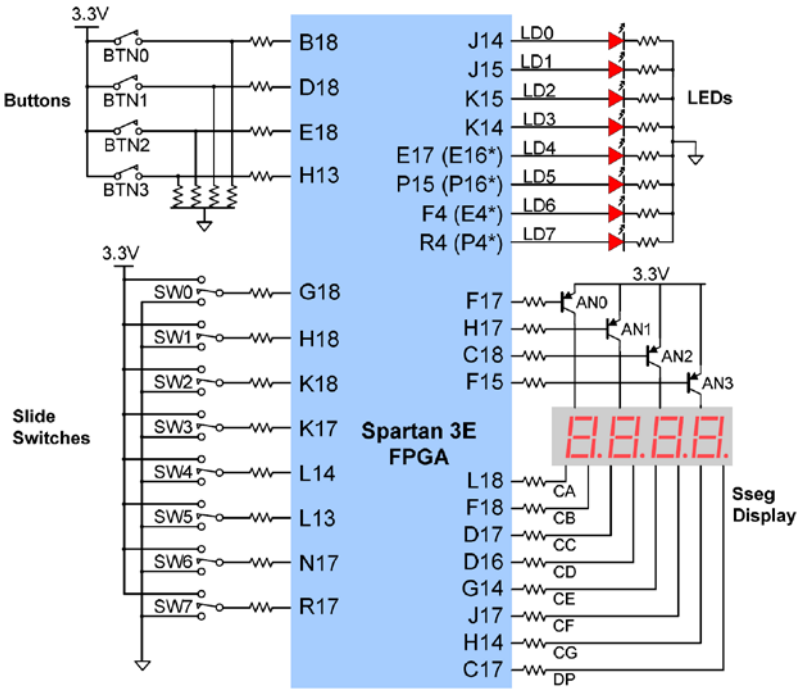


Para añadir el fichero UCF procedemos igual que para añadir los fichero VHDL. El fichero UCF lo debemos crear previamente como texto puro con la extensión *.ucf*

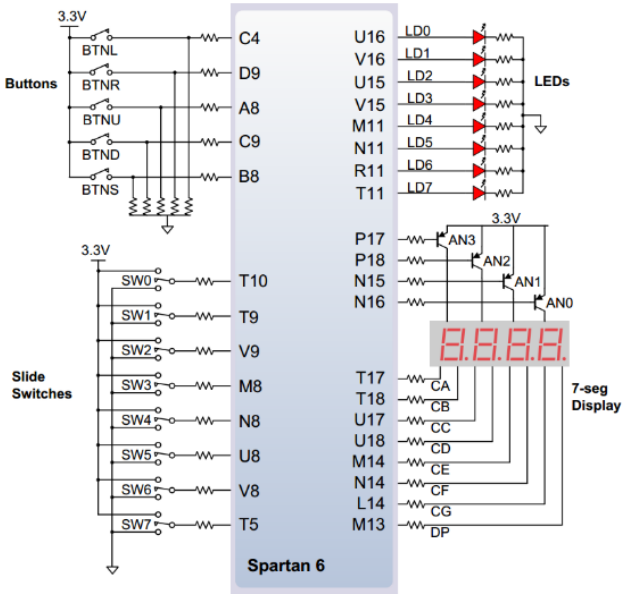
En este fichero debemos indicar qué pines de la FPGA se corresponden con las señales E/S de nuestro diseño y la entrada de reloj. Dependiendo de la placa que usemos, el reloj externo está conectado:

- en la Nexys2, un reloj de 50 MHz en el pin B8
- en la Nexys3, un reloj de 100Mhz en el pin V10

Conexionado de la Nexys2:



Conexionado de la Nexys3:



Ejemplo de fichero UCF para Nexys2:

```
#Reloj
NET "clk_in" LOC = "B8";

#Botones
NET "rst" LOC = "B18";
NET "m5_in" LOC = "E18";
NET "m10_in" LOC = "D18";

#LED
NET "d" LOC = "J14";
NET "ep" LOC = "J15";
```

Ejemplo de fichero UCF para Nexys3:

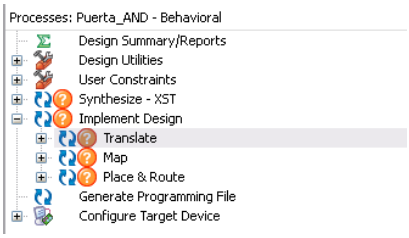
```
#Reloj
NET "clk_in" LOC = "V10";

#Botones
NET "rst" LOC = "C9";
NET "m5_in" LOC = "D9";
NET "m10_in" LOC = "C4";

#LED
NET "d" LOC = "U16";
NET "ep" LOC = "V16";
```

6. Generar el fichero de programación.

En este punto ya hemos descrito nuestro circuito y definido su interfaz e interconexiones físicas. Ahora tenemos que generar el fichero que se transfiere a la FGPA para implementar nuestro diseño. Si seleccionamos el fichero fuente que queremos implementar, en la ventana inferior de la ventana de diseño aparecen los procesos que podemos realizar:

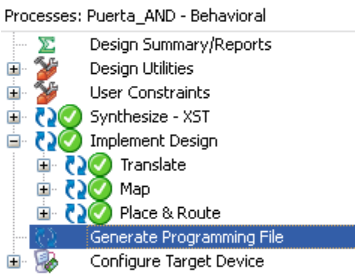


1. **Sintetizar**: Este procedimiento chequea la sintaxis del código VHDL y verifica que todas las conexiones están correctamente. Determina si el circuito es válido e implementable.
   Si tenemos un diseño jerárquico, Mentor incluye la sentencia *LIBRARY Nombre_de_libreria_Mentor;* en el fichero del nivel jerárquico superior. Esto nos dará error en el ISE por lo que debemos borrar esa línea de código cada vez que Mentor regenere dicho fichero (lo podemos hacer desde el propio ISE).

2. **Implementar**: Este proceso se compone a su vez de tres subprocesos.
   a. "Translate" que pasa el código fuente a un lenguaje que se puede implementar en la FPGA.
   b. "Map" que comprueba que nuestro diseño cumple con las especificaciones del dispositivo.
   c. "Place & Route" realiza una serie de algoritmos que optimizan el espacio utilizado en la FPGA, la ubicación y conexionado de los módulos implementados. Como resultado se obtienen una serie de ficheros con la información de dónde se tiene que crear nuestro módulo.

3. **Generar fichero de programación**: Compila todos los procesos en un fichero de programación (*.bit) que podemos transferir a la FPGA.

Podemos realizar los procesos haciendo doble clic sobre cada uno de ellos. A medida que vamos ejecutando los pasos los iconos que aparecen a la izquierda cambian indicando si se han producido o no errores o advertencias.

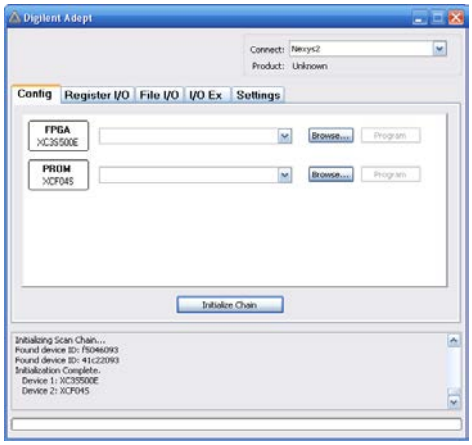


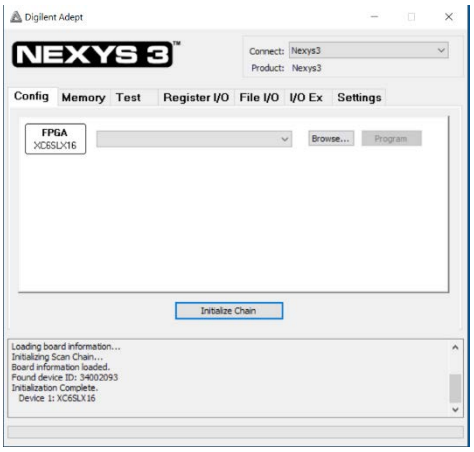
Cuando todos los procesos se han realizado correctamente, podemos generar el fichero de programación. Este paso genera un fichero *.bit* en la carpeta de trabajo del proyecto.

7. Transferir el fichero de programación.

Conectamos la placa Nexys y abrimos la aplicación ADEPT de DIGILENT que nos permitirá transferir el fichero BIT a la placa de desarrollo. Si al abrir la aplicación nos aparece un mensaje de error y nos indica que no encuentra ningún dispositivo conectado es porque la placa no está activada. Debemos conectar la placa con el cable a un puerto USB del PC y utilizar el switch para encender la placa de desarrollo (marcado en la figura con un círculo amarillo).

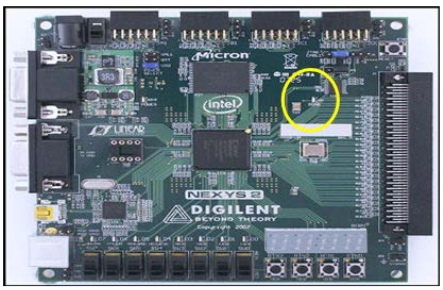
Con la placa correctamente conectada ADEPT reconoce la FPGA automáticamente y dependiendo de la placa Nexys que tengamos conectada nos aparece una de estas ventanas:

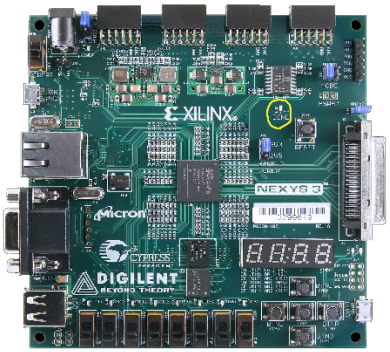




Para el caso de la Nexys2 vemos que aparecen dos dispositivos. La FPGA y una memoria PROM que podemos programar. Nosotros sólo programaremos la FPGA. Para seleccionar el fichero que queremos programar pulsamos "**Browse…**" y le indicamos la ubicación de nuestro fichero .bit generado en el apartado anterior que está en la carpeta de trabajo del proyecto de ISE. Al seleccionar el fichero BIT nos puede aparecer un aviso sobre las señales de reloj al que respondemos que SI.

Finalmente, pulsamos "**Program**" para transferir nuestro diseño a la FPGA. Observa que cuando finaliza la programación se enciende un led amarillo en la placa (DONE) indicando que la transferencia del fichero BIT se ha realizado correctamente (aparece marcado en un círculo amarillo).



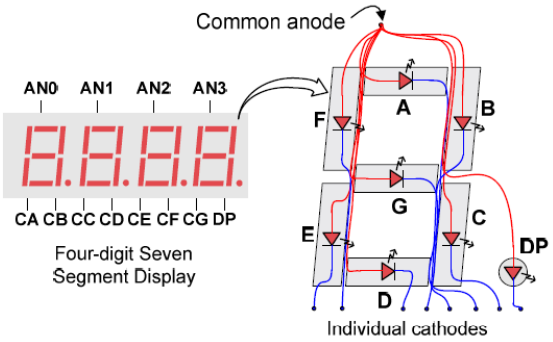


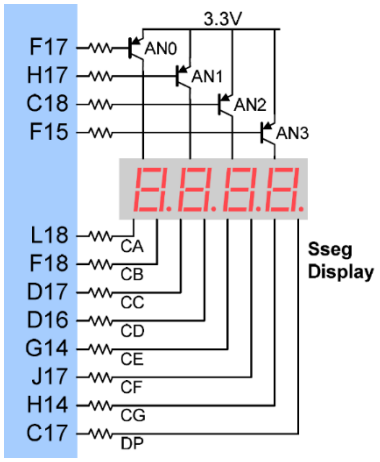
8. Uso de los 4 Displays 7 Segmentos.

Tanto la placa de desarrollo Nexys2 como la Nexys3 constan de 4 displays-7 segmentos que pueden ser utilizados para mostrar datos de salida del sistema. Cada uno de los segmentos se puede iluminar de forma independiente. La placa tiene una arquitectura en ánodo común que consta de:

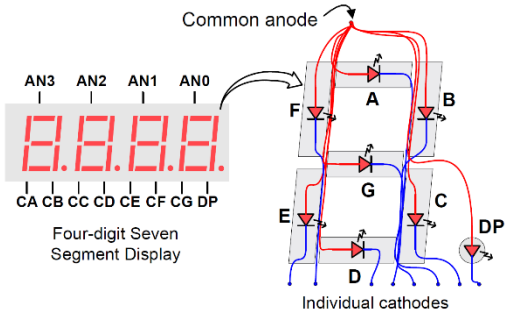
- Un bus de 8 líneas (CA, CB, CC, CD, CE, CF, CG, DP) que determina los segmentos que se iluminan.
- Un bus 4 líneas (AN0, AN1, AN2, AN3) que determina el display deseado.

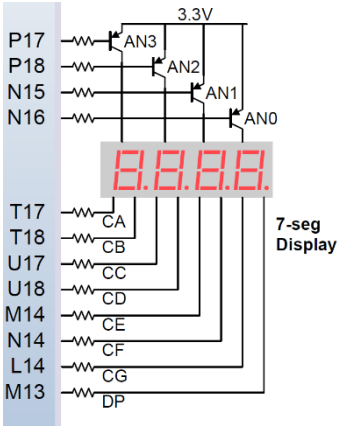
Para la Nexys2 tenemos este conexionado:





Para la Nexys3 tenemos este conexionado:





Tal como se aprecia en ambos circuitos de adaptación entre FPGA y Display, para activar uno de los dígitos, deberemos poner un ***'0'*** en el pin correspondiente. De igual forma, para encender uno de los 7 segmentos, deberemos poner un ***'0'*** en su pin correspondiente.

Fichero UCF para el manejo de display en Nexys2.

```
#Salidas para selector de digito
NET "anodo<0>"    LOC = "F17";   #digito 0 (mas a la derecha)
NET "anodo<1>"    LOC = "H17";   #digito 1
NET "anodo<2>"    LOC = "C18";   #digito 2
NET "anodo<3>"    LOC = "F15";   #digito 3

#Salidas para siete segmentos
NET "siete_segmentos<0>"   LOC = "L18";   #segmento A
NET "siete_segmentos<1>"   LOC = "F18";   #segmento B
NET "siete_segmentos<2>"   LOC = "D17";   #segmento C
NET "siete_segmentos<3>"   LOC = "D16";   #segmento D
NET "siete_segmentos<4>"   LOC = "G14";   #segmento E
NET "siete_segmentos<5>"   LOC = "J17";   #segmento F
NET "siete_segmentos<6>"   LOC = "H14";   #segmento G
```

Fichero UCF para manejo de display en Nexys3.

```
#Salidas para selector de digito
NET "anodo<0>"    LOC = "N16";   #digito 0 (mas a la derecha)
NET "anodo<1>"    LOC = "N15";   #digito 1
NET "anodo<2>"    LOC = "P18";   #digito 2
NET "anodo<3>"    LOC = "P17";   #digito 3

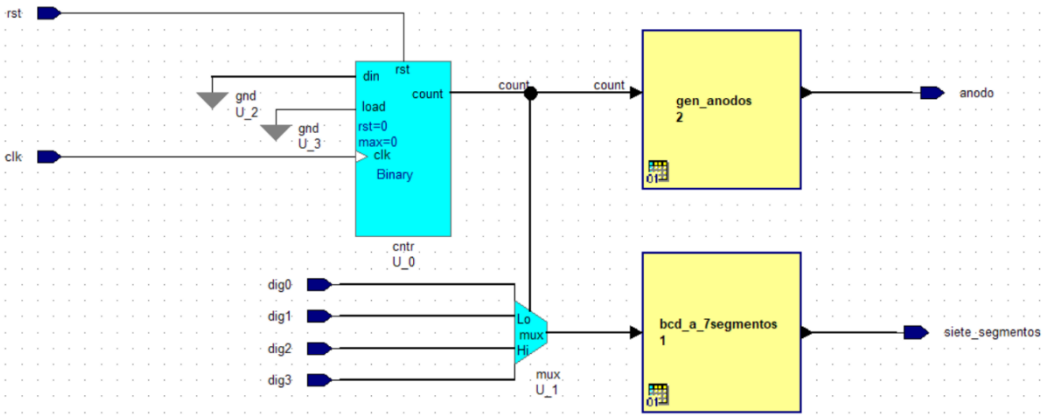
#Salidas para siete segmentos
NET "siete_segmentos<0>"   LOC = "T17";   #segmento A
NET "siete_segmentos<1>"   LOC = "T18";   #segmento B
NET "siete_segmentos<2>"   LOC = "U17";   #segmento C
NET "siete_segmentos<3>"   LOC = "U18";   #segmento D
NET "siete_segmentos<4>"   LOC = "M14";   #segmento E
NET "siete_segmentos<5>"   LOC = "N14";   #segmento F
NET "siete_segmentos<6>"   LOC = "L14";   #segmento G
```

Como los cuatro displays comparten el mismo bus para los cátodos, a priori nos impide tener los cuatro dígitos encendidos simultáneamente. Por este motivo, si queremos mostrar los cuatro dígitos a la vez, es necesario diseñar una lógica que nos permita multiplexar en el tiempo las señales de los cátodos. En otras palabras, hacer que se enciendan y apaguen secuencialmente a una velocidad lo suficientemente alta que sea imperceptible para el ojo humano. Para ello, es suficiente con que se enciendan los cuatro dígitos al menos una vez cada 16 milisegundos = 62.5 Hz.

Dependiendo del tipo de placa que usemos, la FPGA está excitada con un reloj de 50 MHz o de 100Mhz, que es muy superior a la anteriormente indicada, pero es demasiado elevada para permitir el apagado de los leds.

Una frecuencia de trabajo adecuada es entorno a los 9 KHz. Para conseguirla usaremos un divisor de frecuencia que dependerá de la placa que usemos:

- Para Nexys2: 50.000.000 Hz / 80.000 = 6.250 Hz = 6,25KHz
- Para Nexys3: 100.000.000 Hz / 80.000 = 12.500 Hz = 12,5 KHz



Este es un circuito que permite visualizar 4 dígitos en BCD que entran por los buses ***dig0, dig1, dig2, dig3*** y genera como salidas el bus que selecciona el display ***anodo*** y el bus que enciende los display ***siete_segmentos***. Se aprecia el contador de 2 bits, el multiplexor MUX-2 y los generadores de los buses de salida. La entrada de reloj debe ser próxima a los 9 KHz.

9. Restricciones de la señal de reloj en la implementación en FPGA.

La señal de reloj de un diseño debe tener un tratamiento especial distinto de cualquier otra señal. Si no lo hacemos, se producirán retrasos en su propagación por la FPGA y un mal funcionamiento del diseño.

De forma automática el *Xilinx ISE Desing* identificará y tratará de forma correcta la señal de reloj de entrada del sistema. Lo que sí es conveniente es especificar la restricción de la velocidad del reloj de entrada. Esto deberemos especificarlo en el fichero UCF de nuestro diseño.

Ejemplo de fichero UCF para Nexys2:

```
# clk -> Reloj externo de entrada de 50 MHz por PIN B8
NET "clk" LOC = "B8";

# Imponemos a la linea de reloj una restriccion de 50000 KHz = 50 MHz
NET "clk" TNM_NET = sys_clk_pin;
TIMESPEC TS_sys_clk_pin = PERIOD sys_clk_pin 50000 kHz;
```

Ejemplo de fichero UCF para Nexys3:

```
# clk -> Reloj externo de entrada de 100 MHz por PIN V10
NET "clk" LOC = "V10";

# Imponemos a la linea de reloj una restriccion de 100000 KHz = 100 MHz
NET "clk" TNM_NET = sys_clk_pin;
TIMESPEC TS_sys_clk_pin = PERIOD sys_clk_pin 100000 kHz;
```

El problema lo tendremos cuando necesitamos reducir la frecuencia del reloj de entrada generando una nueva señal de reloj de más baja frecuencia. Cuando invoquemos el sintetizador, nos avisará de que tenemos una señal que no es el reloj de entrada pero que va a las entradas de reloj de nuestros módulos. Nuestra nueva señal de reloj de más baja frecuencia no será tratada como de reloj y el diseño puede no funcionar correctamente.

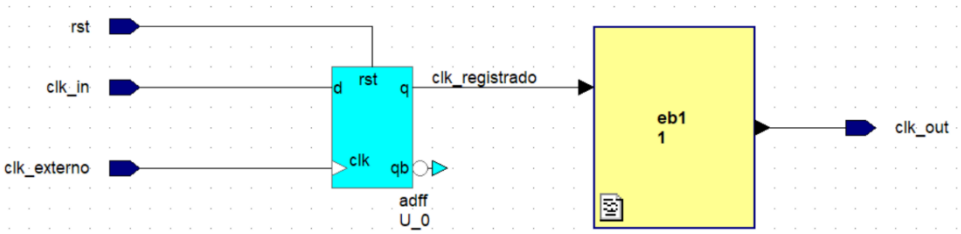
Para solucionar el problema debemos usar un elemento propio de la librería Xilinx: el *BUFG*. Nuestra nueva señal de reloj la deberemos introducir en el *BUFG* y de esta forma Xilinx sabrá que es una señal de reloj y la trazará dentro de la FPG de forma adecuada.

El elemento *BUFG* está contenido en la librería propia de Xilinx *LIBRARY UNISIM* que no tenemos disponible ni en Mentor ni en ModelSim. Para simplificar el uso de este elemento se ha creado un elemento que podremos usar en nuestros diseños: el elemento *buffer_reloj* y se ha incluido la librería UNISIM (aunque vacía) para ModelSim de forma que nuestros diseños que lo contengan **puedan ser generados** en VHDL aunque **no simulados**.

Por tanto, en nuestros diseños simplemente debemos incluir el módulo *buffer_reloj* y **podremos generar los ficheros** necesarios para programar la FPGA, pero la parte del diseño que lo contenga (típicamente la que llamamos *interfaz*) **no podrá simularse**.

Para evitar problemas con el módulo divisor de frecuencia de Mentor, el elemento *buffer_reloj* incluye a su entrada un registro. Sin este elemento, Xilinx daría problemas dependiendo de la estructura del divisor de reloj.

Este es el diseño de *buffer_reloj* donde ***clk_in*** es nuestra señal de reloj de baja frecuencia y ***clk_out*** es esa misma señal metida por el *BUFG*, que es la que deberemos conectar a las señales de reloj de nuestro diseño. La señal ***clk_externo*** es la señal de reloj de entrada (la de 50MHz ó 100MHz) y ***rst*** es el reset general del sistema.

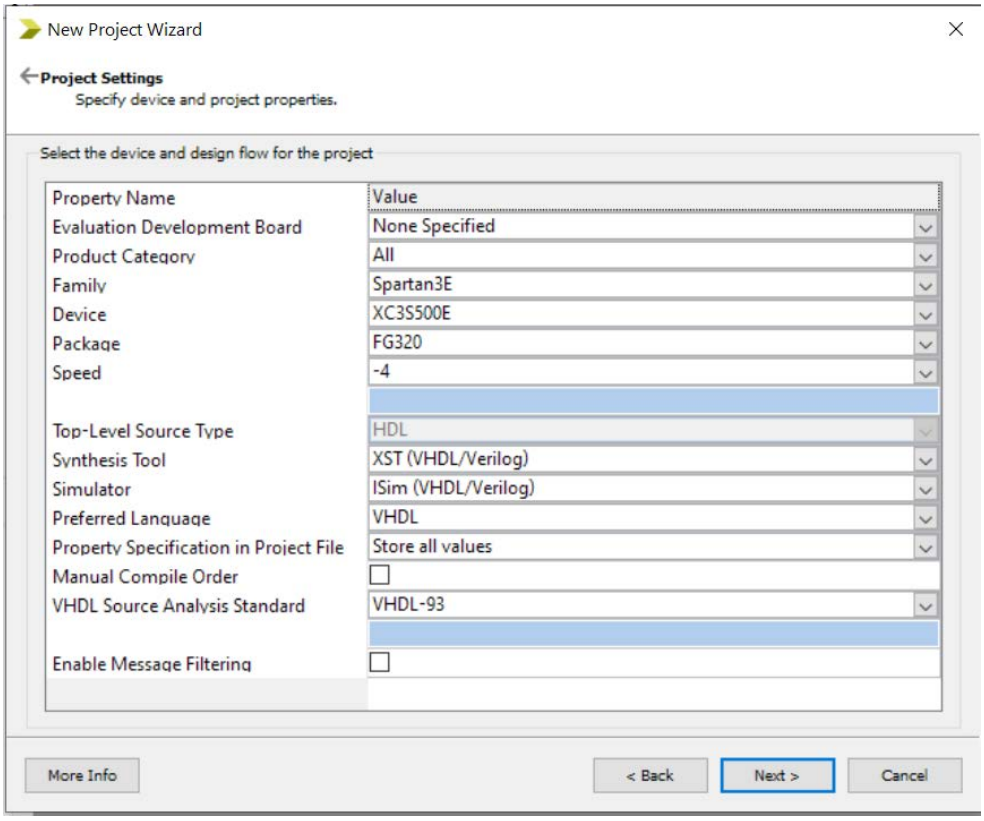


Este es el código VHDL del módulo *eb1*

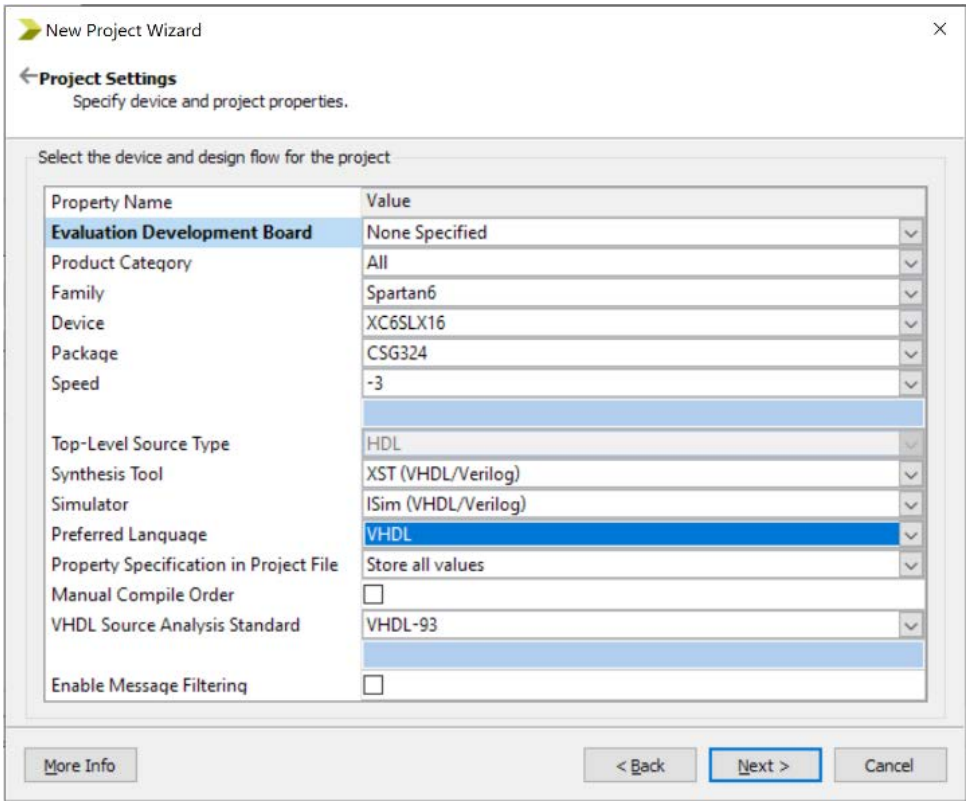
```
BUFG_inst : BUFG
port map (
  O => clk_out          -- Clock buffer output
  I => clk_registrado   -- Clock buffer input
);
```

10. Apendices Gráficos

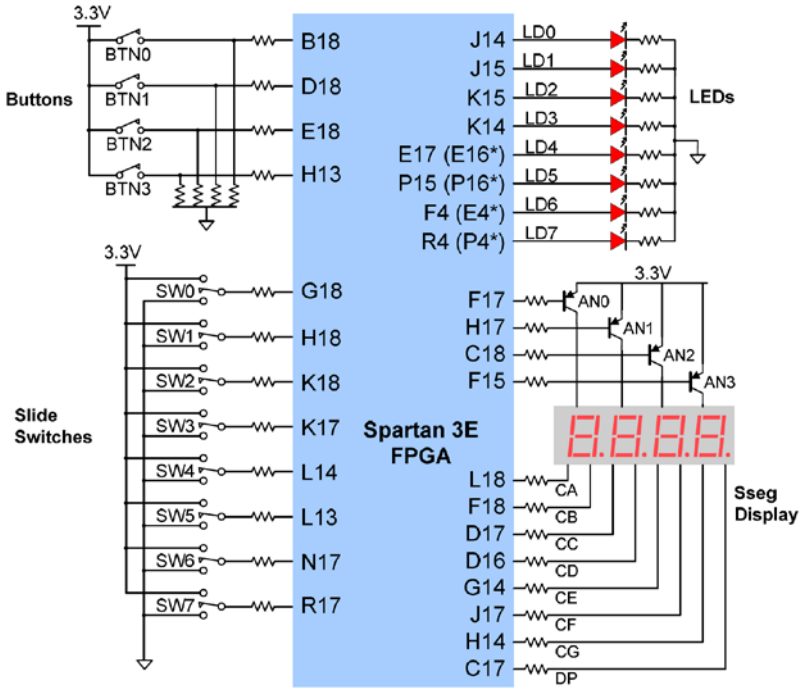
Para Nexys2



Para Nexys 3



Para Nexys2 tenemos un reloj de 50 MHz en el pin B8



Para la Nexys3 tenemos un reloj de 100Mhz en el pin V10

